Section 7.2 Arrays and Pointers

Array definition

- An array is a collection of elements of the same type residing in consecutive memory locations.
  - A collection means there may be more than one element
  - Same type means each element occupy the same size of memory
  - Put in consecutive memory locations means there is no gap between adjacent elements.

Array Analog

- Image a group (aka collection) of students book consecutive hotel rooms.
  - Each student resides in one room (ie, same memory size for each student), once we know the room number (ie, index) of the first student, we know the room numbers (ie. indices) of the rest students.
  - With known address, we can locate the student (ie, value).

Array Analog: II

- a group of students book consecutive hotel rooms.

15	36	21

*arr returns 15, *(arr+1) returns 36, *(arr+2) returns 21

*arr + 1 dereferences arr first, yielding the first element of arr, which is 15. Adding 1 to that result gives 16.

*(arr + 1) applies the dereference operator * to (arr + 1), which is the address of the second element in arr. Thus, *(arr + 1) returns 36.

Why array

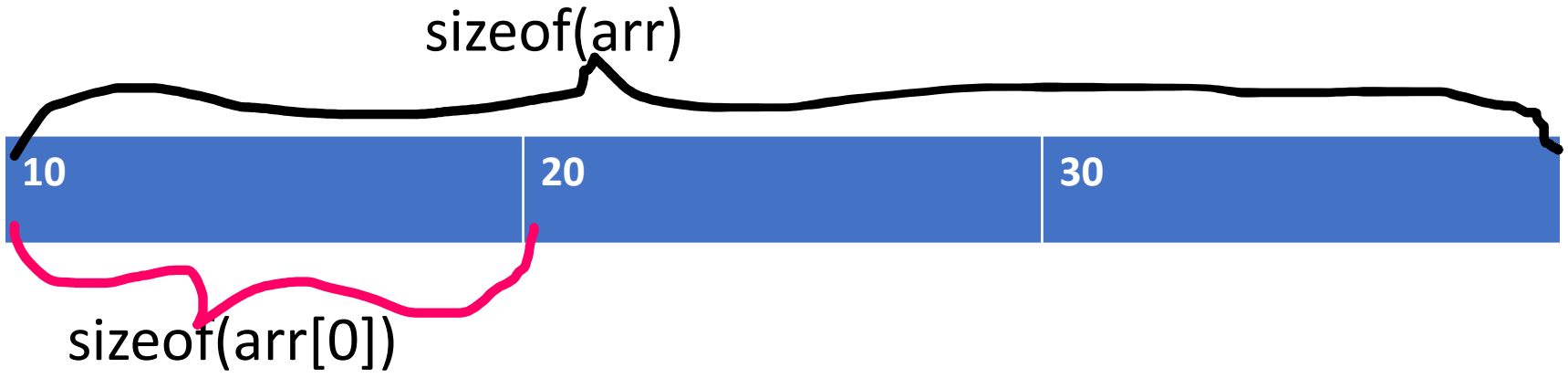
- Pro: Constant time to access each element.
- Con:
  - Need to apply for (not mandate) capacity (maximum number of elements residing in the array) in the beginning and this number cannot be changed later on.
  - Inserting or removing an element requires shifting subsequent elements, except when operation occurs at the end of the array.
- Suitable for elements with no or little change, like the names of states or the words in a dictionary.

The name of an array implies the address of its first element: code link

```
int arr[] = {10, 20, 30};
cout << "address of arr " << arr << endl;
cout << "address of the first element of arr " << &arr[0] << endl;
int size = sizeof(arr) / sizeof(arr[0]);
for (int i = 0; i < size; i++) {
    cout << "address of element indexed at " << i << " is " << &arr[i] << endl;
    cout << "value of element indexed at " << i << " is " << arr[i] << endl;
}
```

Calculate array size

```
int arr[] = {10, 20, 30};
cout << "address of arr" << arr << endl;
cout << "address of the first element of arr " << &arr[0] << endl;
int size = sizeof(arr) / sizeof(arr[0]);
```



Calculate array size: II

- Must be calculated in the the same function where an array is declared or initializes, so the system know the actual memory size of array.

```
double array_average(int arr[]) {
    int size = sizeof(arr) / sizeof(arr[0]);   //WRONG
}
```

Constant access time for each element

- Address of arr[i], while 0 <= i < size, where size is the capacity of the array, is calculated as follows, each int takes 4 bytes.

  arr + i is address_of_arr + i * size_of_a_single_element_in_the_array.

```
address of arr0x7ffeea4919cc
address of the first element of arr 0x7ffeea4919cc
address of element indexed at 0 is 0x7ffeea4919cc
value of element indexed at 0 is 10
address of element indexed at 1 is 0x7ffeea4919d0
value of element indexed at 1 is 20
address of element indexed at 2 is 0x7ffeea4919d4
value of element indexed at 2 is 30
```

Array vs pointer

- Given an array called arr with (size) many elements.
- Given index i, where 0 <= i and i < size.
- arr is the name of the array, its value is also the address of the first element of the array. That is, arr is the same as &arr[0].

Array vs Pointer: II

- *arr is the first element of the array, that is, *arr is *(&arr[0]) – the guy whose address is &arr[0] -- which is arr[0].
- arr+i is the address of arr[i]. This is, arr + i is the same as &arr[i].
- Then *(arr + i) is *(&arr[i]), that is, *(arr + i) is arr[i].
- <u>When defining a function</u>, parameter int arr[] is the same as int* arr. Need to pass size of the array as well.

Statically Allocated Array in a Function Cannot be Returned to its Caller

```
int* returnArray(int size) {
    const int CAPACITY = size;

    int arr[CAPACITY];
    for (int i = 0; i < CAPACITY; i++) {
        arr[i] = i;
    }

    return arr; //WRONG, memory in statically allocated array,
    //allocated from stack, is reclaimed after the function is finished.
    //Such an array declared in a callee cannot be returned to its caller.
}
```

How to Return an Array Created inside a function?

- To return an array created inside a function, you must use Dynamic Memory Allocation.
- This places the data on the **Heap**, which persists until you manually delete it.
- Dynamic Allocated Memory is introduced in Section 7.4.

The Library Analogy of Memory Management

- **Digitally Licensed E-books (Stack Allocation):** Like an expiring license, once the **function scope** — the due date — is reached, the e-book is **automatically** reclaimed by the system. No action is required from the borrower.
- **Self-Checkout Physical Books (Heap Allocation):** You borrow a book from the shelf using the **new** operator, and it remains yours until you manually return it using the **delete** operator. If you never return it, a lost book — **memory leak** — occurs.